

Шакиров Тимур Эдуардович

Инженер. Языковые модели в продакшене: шлюзы, биллинг, конвейеры контента. Последний год - e-commerce платформы на React и GraphQL.

Санкт-Петербург · удалённо +7 981 691-94-36 edtimyr@gmail.com
github.com/TimaxLacs github.com/TimaxHack github.com/timaxlax npmjs.com/~timax

Работа распределена между тремя аккаунтами: основной объём кода в deep-assistant написан под TimaxHack.

НАВЫКИ

ЯЗЫКИ	Python, TypeScript, JavaScript
ФРОНТЕНД	React, Next.js, Vite, Tailwind, Capacitor (сборки под iOS и Android)
БЭКЕНД	Node.js, Express, FastAPI, GraphQL (Hasura), REST, PostgreSQL / PostGIS, Redis, MongoDB, SQLite, MinIO
ИИ	маршрутизация между провайдерами и фолбэк, биллинг токенов, RAG и MCP, очереди генерации текста и изображений, reasoning-конвейеры, распознавание и синтез речи, LangChain
ПРОД	Docker, nginx, systemd, Linux, Git, CI, настройка DNS и почты

ОПЫТ РАБОТЫ

Deep.Assistant - агрегатор нейросетей

2024-2025

Сооснователь, backend и DevOps · команда из трёх человек

Коммерческий продукт: чат с моделями, генерация изображений, музыки и голоса, API-шлюз, мини-приложения в Telegram и VK.

LLM-шлюз - github.com/deep-assistant/api-gateway

Спроектировал и написал первую версию; вырос из моего личного репозитория resale-chatgpt-azure в инфраструктуру, через которую идут все запросы продукта. Основной автор кода: HTTP-сервер, конфигурация моделей, менеджер токенов, слой работы с базой.

- Около 37 моделей от шести поставщиков за одним OpenAI-совместимым интерфейсом: OpenAI напрямую, OpenRouter, DeepInfra, DeepSeek и реселлеры.
- Таблица деградации вместо простого ретрая.** Для каждой модели прописана упорядоченная цепочка из 12-16 замен: если провайдер недоступен, запрос уходит на ближайшую по смыслу модель, а не просто падает. Замены подбирал вручную, чтобы ответ оставался пригодным для того же сценария.
- Единый счёт поверх несопоставимых прайсов.** У каждой модели свой коэффициент пересчёта токенов во внутреннюю валюту - от 1.7 у крупных моделей до 40 у мелких. Пользователь видит один баланс, продукт считает себестоимость запроса и наценку.
- Переводы между счётами без СУБД.** Хранилище файловое, поэтому сделал эксклюзивные блокировки с распознаванием зависших, запись через временный файл с обратной вычиткой и ротацией резервных копий, а также восстановление незавершённых переводов при старте.
- Авторизация по ключам, общий баланс для бота и внешнего API.

Веду репозиторий: принимаю и ревьюю изменения второго разработчика.

Telegram-бот - github.com/deep-assistant/telegram-bot

Мультимодельный ассистент на aiogram, над которым работали четверо. Мои части:

- **Переводы внутренней валюты между пользователями.** Диалог на конечном автомате: получатель, сумма, подтверждение, отмена, история операций. Написан с нуля целиком.
- Сервис синхронизации пользователей: ленивая догрузка профиля при первом обращении и массовая сверка с ограничением параллелизма, чтобы не упереться в лимиты Telegram.
- Режим транскрибации аудио с отдельным коэффициентом тарификации и пользовательские системные сообщения.
- Поведение в групповых чатах: бот отвечает только на обращения к себе, разбирает упоминания в подписях к фото и документам. Выгрузка истории диалога файлом.
- Оплата через Telegram Stars и интеграция внешнего рекламного API за фича-флагом.
- Разделение окружений TEST и PROD, docker-compose под оба, асинхронные проверки доступности провайдеров.

Бот [@DeepGPTBot](#), канал [@gptDeep](#) - 1 553 подписчика.

Скорость вывода моделей в прод

Подключал reasoning-модели в течение **10-15 дней** после публичного релиза, вместе с тарификацией и цепочками фолбэка: o1-preview и o1-mini - 27.09.2024, DeepSeek-R1 - 30.01.2025.

Продуктовая работа

Завёл 14 продуктовых задач по боту: умный приоритет провайдеров, режим экспресс-ответов, напоминания, сохранение прошлых контекстов, подключение новых моделей. Задачу про умный приоритет API коллега реализовал через неделю.

Е-commerce платформа (проект заказчика, под NDA)

декабрь 2025 - настоящее время

Контрактный инженер: фронтенд, гео, мобильные сборки, смежный DevOps

Маркетплейс с каталогом и картами: монорепозитарий фронтенда и восьми backend-сервисов. React и TypeScript, Hasura GraphQL, PostgreSQL с PostGIS, Redis, MinIO, nginx, Docker, Capacitor. Клиентское приложение веду один; на бэкенде - платежи, баланс и обработчики событий. Общие библиотеки монорепозитария, схему базы и CI делает другой разработчик. Детали продукта под NDA, поэтому ниже - модули и инженерные решения.

Витрина, каталог и поиск

Каталог, карточка товара, редактирование позиций продавцом, подборка на главной. Поиск с автодополнением и набор фильтров - этот слой писал целиком, вместе с утилитами разбора поисковой строки и активными фильтрами, которые можно снимать по одному. Отдельно - библиотека UI-компонентов, на которой собран весь интерфейс.

Регистрация, вход и профиль

Регистрация нескольких типов аккаунтов со своими схемами валидации, вход, сброс пароля, многофакторное подтверждение, хранилище регистрационных токенов на стороне сервера. Профиль с вкладками и загрузкой аватара.

Двухязычность и автоперевод карточек

Интерфейс на русском и английском: словари вынесены из компонентов и типизированы, локализованные поля сущностей достаются одной утилитой. Перевод карточек не ручной - есть хук автоперевода, серверный роут перевода и CLI-скрипт, который прогоняет каталог пачкой. Добавление языка не требует правки разметки.

Кошелёк, оплата и заказы

Единственная вертикаль, где писал и клиент, и сервер: сервисы баланса и платежей, роуты покупок и ставок, курс валют, интеграция внешнего эквайринга, страница кошелька, оформление и страница заказа. Смена статуса сделки и новые отклики обрабатываются событийными обработчиками на стороне бэкенда.

Мобильные приложения на Capacitor

Из той же кодовой базы собираются приложения под iOS и Android: проекты Xcode и Gradle, доступ к геолокации, платформенная навигация, определение платформы в рантайме, сервис-воркер для офлайн-кеша. Прошёл требования App Store: манифест приватности и механика жалоб и блокировок для пользовательского контента – без неё приложение с UGC не публикуют.

Конвейер наполнения каталога

Спроектировал схему загрузки товаров от внешних поставщиков. Источники спрятаны за общим контрактом, поэтому новый поставщик добавляется без изменения остального кода. Дубликаты отсекаются по трём признакам: внешний идентификатор, штрихкод и отпечаток характеристик товара.

Главное решение – нейросети работают *после* сбора данных, отдельными очередями: воркер текстовых описаний RU и EN и воркер изображений. Сбор не блокируется генерацией, а упавший воркер оставляет товар на витрине с исходными характеристиками, описание дописывается позже. Характеристики поставщика модель не переписывает, чтобы не портить фактические данные.

Гео-модуль – отдельный продукт внутри платформы

Единственный разработчик модуля, около 11 700 строк. Единый интерфейс поверх трёх картографических провайдеров: Яндекс, Google и 2ГИС. Архитектура на адаптерах – React-компоненты карты, маркера и зоны ничего не знают о конкретном провайдере; движки маркеров и зон принимают команду и переадресуют её адаптеру, а тот переводит унифицированные данные в вызовы своего SDK. Смена провайдера – это смена одного параметра, остальной код приложения не трогается.

- **Геометрия зон, около 1 700 строк:** буферные зоны вдоль маршрута с плавными переходами к круговым зонам маркеров через квадратичные кривые Безье, объединение полигонов с поддержкой отверстий и мультиполигонов, несколько режимов объединения, валидация геометрии и кэширование расчётов.
- Перетаскиваемые маркеры, пользовательские зоны, изменения с карты поднимаются в состояние приложения через колбэки.
- Геокодинг вынесен на серверный роут, чтобы ключи API не попадали в браузер.
- Хранение геометрии в PostGIS, поиск по радиусу, мультиязычность, Docker, смоук-тесты и документация по выносу модуля в самостоятельное приложение.

Модуль отделяем от платформы: описал, что нужно вынести, чтобы он запускался самостоятельным приложением.

Инфраструктура и выкладка

Написал инструкцию развёртывания, по которой проект поднимается с нуля. Backend-сервисы переведены на systemd, контейнеры получили политику перезапуска, фронт отдаётся через nginx. Провёл миграцию базы на PostGIS – без неё пространственные запросы гео-модуля упирались в перебор на стороне приложения.

ManyAir – платформа международных денежных переводов

август 2025 – май 2026

Проект заказчика: фронтенд и смежный бэкэнд

Микросервисная система вокруг Hasura GraphQL: router, обработчик событий, рассылка сообщений, Telegram-бот, генератор документов, сервис курсов валют; PostgreSQL, Redis, MinIO, уведомления по SMS и email.

- **Аукционная механика:** публичные лоты и адресные предложения, ролевая модель компаний и их сотрудников.
- **Устойчивость доставки уведомлений:** если канал недоступен, статус честно возвращает «не отправлено», а приглашение не помечается доставленным при неработающей рассылке.
- **Изоляция данных между ролями:** сброс кеша запросов при смене пользователя, ограничение подписок на детали сделки до появления безопасного превью.

- Удаление устаревшего домена приглашений целиком: таблицы, права Hasura, роуты, обработчики уведомлений, сгенерированные типы и сценарии ручного тестирования.
- Миграция корпоративной почты проекта на связку Selectel MTA и Яндекс 360 с перенастройкой DNS.

Matreshka - AI/e-commerce платформа

2026

Проект заказчика: фронтенд и запуск публичного сайта · matreshka.club

Позиционирование платформы «Where AI + Data Meets eCommerce»: биржевое ценообразование, агрегация спроса, голосовые агенты. React, Vite, TypeScript, Tailwind, shadcn/ui; фирменный стиль и варианты логотипа; деплой на nginx с SSL, настройка DNS и корпоративной почты на домене. Второй проект для того же заказчика.

Такси 628 - биржа заказов

2024

Fullstack, заказная разработка · taxi628.ru

Telegram-бот биржи заказов для таксопарка: оператор размещает заказ, водители видят и подтверждают его в реальном времени. Внедрён у заказчика, заменил ручную раздачу смен.

ИНЖЕНЕРНЫЕ КЕЙСЫ С ИИ

Reasoning-конвейер поверх обычных чат-моделей

апрель 2025

github.com/TimaxLacs/test-resorning

Многоступенчатый reasoning-конвейер на моделях без встроенного режима рассуждений. Написан в апреле 2025, когда такой режим ещё не был стандартной функцией продуктов.

- Пять стадий. Сначала модель предсказывает, чего на самом деле хочет пользователь, потом решает задачу по шагам. Дальше она играет антагониста и пытается опровергнуть собственный ответ, правит решение по найденным возражениям и только затем собирает итоговый ответ.
- Вариант на LangChain: Tree of Thoughts с тремя независимыми ветками рассуждений плюс самосогласованность - критическая оценка всех веток и синтез лучшей.
- Полная цепочка рассуждений выгружается файлом, чтобы решение можно было проверить.

Ни одна стадия не опирается на встроенный режим рассуждений: весь эффект достигается оркестрацией промптов на обычных чат-моделях.

Контент-конвейер с ИИ-модерацией

2026

github.com/TimaxLacs/ai-post-generator

Асинхронный конвейер: одна модель пишет черновик, вторая проверяет его отдельным вызовом, замечания возвращаются на правку - до трёх кругов, после чего пост уходит в публикацию. Состояние хранится в SQLite, а не в файлах, чтобы конвейер переживал падение процесса; публикаторы вынесены в отдельные стратегии под Telegram, VK и MAX.

Генерация отделена от проверки, состояние переживает падение процесса - та же схема применима к описаниям товаров на нескольких языках.

Память для агентов: RAG и MCP

2026

github.com/TimaxLacs/lightrag-mcp

Развернул self-hosted LightRAG - граф плюс векторы - и написал собственный MCP-сервер для интеграции с редактором: режимы поиска mix, hybrid и local, ранкинг, цитирование источников. Около 250 строк на

Node.js и 170 строк тестов, конфигурация валидируется через zod.

Голосовой стек

2025

github.com/TimaxLacs/InnerEcho

Голосовой ассистент целиком на локальной машине: распознавание речи, языковая модель, синтез с клонированием голоса, автонастройка аудиоустройств. Отдельно сравнивал модели синтеза речи на русском корпусе. Исследовательский прототип, не прод.

Собственная методология работы с ИИ-агентами

2026

github.com/TimaxLacs/cursor-agent-workflows

Написал набор агентных процессов: конвейер безопасного внесения изменений из пяти специализированных агентов (анализ, план, параллельная реализация, проверка, разбор итогов), пятифазный бутстрап нового проекта, процесс глубокого исследования. Около **1 900 строк** спецификаций, которыми пользуюсь ежедневно.

Ключевые решения: анализатор запечатывает границы задачи, и реализация не может выйти за них; независимые куски работы идут параллельными потоками с отдельной проверкой на коллизии; упавший поток переигрывается сам по себе; правки в инструкции никогда не применяются молча – их предлагает агент, а решает человек.

ЭКОСИСТЕМА DEEP.FOUNDATION

Ассоциативная модель данных: участие с 2024 года

2024-2025

github.com/deep-foundation · участник организации

Deep.Foundation строит хранилище, где единственная сущность – связь: кортеж из идентификатора, типа и ссылок на другие связи. Связь может ссылаться на связь, поэтому одной таблицей описываются и данные, и схема, и обработчики. Я пришёл туда как пользователь платформы и остался писать инструменты вокруг неё.

- **deep-files-sync** - проекция графа связей на файловую систему. Мой пакет, 13 версий. Связи раскладываются в каталоги, значение лежит в текстовом файле рядом с объектом, а вложенность папок выражает отношения: путь по директориям изоморфен запросу к базе. Правки с обеих сторон синхронизируются, подписка на изменения материализуется файлом, удаление файла означает отписку. Смысл в том, чтобы содержимое базы можно было читать и править обычными инструментами – grep, редактором, git. github.com/TimaxLacs/deep-files-sync
- **Пакеты ИИ-тулинга в официальном скоупе.** Выпускал версии в @deep-foundation/ai, ai-models, ai-templates, ai-tasks, 4o-bot, chatgpt-azure-deep, function_generation, tests_generation. Такой пакет – не библиотека, а выгрузка настроенного графа связей, которая разворачивается в чужом инстансе.
- **Диалоги бота как граф.** Реализовал запись переписки в ассоциативное хранилище: диалог, сообщение, автор и ответ выражены типами связей, без отдельных таблиц под каждую сущность. Решение прожило в проде около шести недель и было заменено на файловое хранилище – сложность не окупилась.
- Соавтор русскоязычной теоретической документации по модели, около 1 000 строк. Завёл порядка 36 issue – в основном баг-репорты по развёртыванию платформы, с трейсбеками.

Честная граница: в ядро deepLinks я кода почти не писал – там мой вклад это обновление зависимостей и один тест.

Пакеты синтеза речи и утилиты

2024-2025

npmjs.com/~timax

- **xtts-v2-js** - обёртка над XTTS-v2 с клонированием голоса на 17 языках: сама поднимает Python-окружение, чтобы пакет ставился как обычная зависимость Node.js. 7 версий.
- **zonosjs** и **zonos-client** - клиент к локальному серверу синтеза речи Zonos, 10 версий суммарно.
- **@timax/passport-authorization** и связанные пакеты - регистрация и подтверждение по email.

tg-runner - оркестрация ботов

январь 2026

Комплект из четырёх компонентов

Оркестратор запуска ботов на конечном автомате с Redis, воркер, поднимающий ботов в Docker, публикуемый SDK для своих воркеров и CLI. Docker Compose, pytest, документация.

github.com/TimaxLacs/ - [tg-runner-orchestrator](#) · [tg-runner-worker](#) · [tg-runner-worker-sdk](#) · [tg-runner-cli](#)

Вклад в чужие проекты

2024-2026

- [deep-foundation/chatgpt-telegram-bot](#) - привёл проект к воспроизводимой сборке: Dockerfile, workflow сборки образа в GitHub Actions и документация запуска. Влило через pull request.
- Форк [hasux](#) - голосовой слой: транскрибация через Whisper, выбор аудиоустройств, память диалога, прерывание синтеза речи.
- Всего **10 принятых pull request** в чужие репозитории; issue завожу с трейсбеками и шагами воспроизведения.

ПРОЕКТЫ

Боты для мессенджера MAX

2026

github.com/TimaxLacs/max-bot

Мультимодальная обработка фото нейросетью, квота на пользователя, Docker, интеграция с асинхронным ИИ-воркером и запасной сценарий при недоступности бэкенда.

vk-tg-parser

2026

github.com/TimaxLacs/vk-tg-parser

pip-пакет: единый парсер Telegram через MTProto и VK через API с общей моделью данных и мониторингом в реальном времени.

Автокомментер по хэштегам

2026

Комментирование по хэштегам в Telegram и VK с ограничением частоты и дедупликацией. Реальный прогон в мае 2026.

ai-lector

2025

github.com/TimaxLacs/ai-lector

Бот, перерабатывающий аудиолекции под нужный формат. Собрал локальный Telegram Bot API server через

смаке, чтобы обойти лимит на размер файла.

anti-spam-ai

2025

Антиспам-бот: модель через OpenRouter выставляет сообщению спам-балл от 1 до 10.

Orbiteer

2025

github.com/TimaxLacs/Orbiteer

Анализатор рисков космических миссий: расчёт вероятности столкновений, 3D-визуализация орбит на three.js, API на Sanic.

Сайты на React и Vite

2026

github.com/TimaxLacs/HSK-Grom и [chkalov-park](#): календарь событий, карта, деплой на GitHub Pages.

Техническая статья про изоляцию Docker-провайдеров

2024

github.com/TimaxLacs/articles

Прошла внешнее ревью в четырёх pull request.

РАННИЕ РАБОТЫ, ДЛЯ КОНТЕКСТА

GEFFEST - перчатка-манипулятор

2021-2022

github.com/TimaxHack/GEFFEST

Датчики считывают положение руки через кватернионы, обмен между перчаткой и манипулятором по радио, сглаживание движения сервоприводов, своя пайка макетных плат. Плюс Android-приложение голосового управления на Java и Vosk.

Ранние боты и компьютерное зрение

2021

[Бот-библиотека](#) - карта библиотек с нужной книгой, командный проект. Бот замерзания Онежского озера - определение уровня замерзания по кадрам с уличной камеры компьютерным зрением.

Запуски, которые не взлетели

2023-2025

ArtApollo - студия, где заказы отрисовывают нейросети. Собрал команду до 30 человек, взял первые платные заказы, выполнил корпоративный заказ на 12 человек. Проект закрыт, команда распалась, итоговый доход меньше 5 000 рублей.

Cyber Vestis - прототип умной одежды с климат-контролем: ПО для сенсорного дисплея, телеметрия температуры тела. Дошли до рабочего прототипа, до инвестиций не дошли.

Обе истории закончились ничем, но дали опыт сборки команды и доведения прототипа до демонстрации.